

CORE CONCEPTS

Work with sessions

 Copy page

How sessions persist agent conversation history, and when to use `continue`, `resume`, and `fork` to return to a prior run.

A session is the conversation history the SDK accumulates while your agent works. It contains your prompt, every tool call the agent made, every tool result, and every response. The SDK writes it to disk automatically so you can return to it later.

Returning to a session means the agent has full context from before: files it already read, analysis it already performed, decisions it already made. You can ask a follow-up question, recover from an interruption, or branch off to try a different approach.

❗ Sessions persist the **conversation**, not the filesystem. To snapshot and revert file changes the agent made, use [file checkpointing](#).

This guide covers how to pick the right approach for your app, the SDK interfaces that track sessions automatically, how to capture session IDs and use `resume` and `fork` manually, and what to know about resuming sessions across hosts.

Choose an approach

How much session handling you need depends on your application's shape. Session management comes into play when you send multiple prompts that should share context. Within a single `query()` call, the agent already takes as many turns as it needs, and permission prompts and `AskUserQuestion` are **handled in-loop** (they don't end the call).

Multi-turn chat in one process	<code>ClaudeSDKClient</code> (Python) or <code>continue: true</code> (TypeScript). The SDK tracks the session for you with no ID handling.
Pick up where you left off after a process restart	<code>continue_conversation=True</code> (Python) / <code>continue: true</code> (TypeScript). Resumes the most recent session in the directory, no ID needed.
Resume a specific past session (not the most recent)	Capture the session ID and pass it to <code>resume</code> .
Try an alternative approach without losing the original	Fork the session.
Stateless task, don't want anything written to disk	Set <code>persistSession: false</code> (TypeScript only). The session exists only in memory for the duration of the call. In Python, set <code>CLAUDE_CODE_SKIP_PROMPT_HISTORY</code> in the <code>env</code> option to suppress transcript writes instead.

Continue, resume, and fork

Continue, resume, and fork are option fields you set on `query()` (`ClaudeAgentOptions` in Python, `Options` in TypeScript).

Continue and **resume** both pick up an existing session and add to it. The difference is how they find that session:

- **Continue** finds the most recent session in the current directory. You don't track anything. Works well when your app runs one conversation at a time.
- **Resume** takes a specific session ID. You track the ID. Required when you have multiple sessions (for example, one per user in a multi-user app) or want to return to one that isn't the most recent.

Fork is different: it creates a new session that starts with a copy of the original's history. The original stays unchanged. Use fork to try a different direction while keeping the option to go back.

Python: `ClaudeSDKClient`

`ClaudeSDKClient` handles session IDs internally. Each call to `client.query()` automatically continues the same session. Call `client.receive_response()` to iterate over the messages for the current query. Use the client as an async context manager so connection setup and teardown are handled for you, or call `connect()` and `disconnect()` manually.

This example runs two queries against the same `client`. The first asks the agent to analyze a module; the second asks it to refactor that module. Because both calls go through the same client instance, the second query has full context from the first without any explicit `resume` or session ID:

```
ClaudeAgentOptions,  
AssistantMessage,  
ResultMessage,  
TextBlock,  
)  
  
def print_response(message):  
    """Print only the human-readable parts of a message."""  
    if isinstance(message, AssistantMessage):  
        for block in message.content:  
            if isinstance(block, TextBlock):  
                print(block.text)  
    elif isinstance(message, ResultMessage):  
        cost = (  
            f"${message.total_cost_usd:.4f}"  
            if message.total_cost_usd is not None  
            else "N/A"  
        )  
        print(f"[done: {message.subtype}, cost: {cost}]")  
  
async def main():  
    options = ClaudeAgentOptions(  
        allowed_tools=["Read", "Edit", "Glob", "Grep"],  
    )  
  
    async with ClaudeSDKClient(options=options) as client:  
        # First query: client captures the session ID internally  
        await client.query("Analyze the auth module")  
        async for message in client.receive_response():  
            print_response(message)  
  
        # Second query: automatically continues the same session  
        await client.query("Now refactor it to use JWT")  
        async for message in client.receive_response():
```

Each query prints the agent's text response followed by a status line from the result message, such as `[done: success, cost: $0.0042]` .

See the [Python SDK reference](#) for details on when to use `ClaudeSDKClient` vs the standalone `query()` function.

TypeScript: `continue: true`

The TypeScript SDK doesn't have a session-holding client object like Python's `ClaudeSDKClient` . Instead, pass `continue: true` on each subsequent `query()` call and the SDK picks up the most recent session in the current directory. No ID tracking required.

This example makes two separate `query()` calls. The first creates a fresh session; the second sets `continue: true` , which tells the SDK to find and resume the most recent session on disk. The agent has full context from the first call:

```
try {
  for await (const message of query({
    prompt: "Analyze the auth module",
    options: { allowedTools: ["Read", "Glob", "Grep"] }
  ))) {
    if (message.type === "result" && message.subtype === "success") {
      console.log(message.result);
    }
  }
} catch (error) {
  // A single-shot query() throws after yielding an error result,
  // so the follow-up query below still runs.
  console.error(`Session ended with an error: ${error}`);
}

// Second query: continue: true resumes the most recent session
for await (const message of query({
  prompt: "Now refactor it to use JWT",
  options: {
    continue: true,
    allowedTools: ["Read", "Edit", "Write", "Glob", "Grep"]
  }
  ))) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```

❗ The experimental **V2 session API**, which provided `createSession()` with a `send / stream` pattern, was removed in TypeScript Agent SDK 0.3.142. Use the `query()` function and the session options described on this page instead.

Resume and fork require a session ID. Read it from the `session_id` field on the result message (`ResultMessage` in Python, `SDKResultMessage` in TypeScript), which is present on every result regardless of success or error. In TypeScript the ID is also available earlier as a direct field on the init `SystemMessage` ; in Python it's nested inside `SystemMessage.data` .

```
async def main():
    session_id = None

    try:
        async for message in query(
            prompt="Analyze the auth module and suggest improvements",
            options=ClaudeAgentOptions(
                allowed_tools=["Read", "Glob", "Grep"],
            ),
        ):
            if isinstance(message, ResultMessage):
                session_id = message.session_id
                if message.subtype == "success":
                    print(message.result)
    except Exception as error:
        # A single-shot query() raises after yielding an error result. If the
        # failure was an error result, the loop above already captured session_id.
        # connection or process failures yield no result message, so session_id is None.
        print(f"Session ended with an error: {error}")

    print(f"Session ID: {session_id}")
    return session_id

session_id = asyncio.run(main())
```

When the query completes, the script prints the agent's response followed by a line such as `Session ID: 5b3f2c1a-8d4e-4f6b-9a7c-2e1d0f9b8a6c` . In the next sections, you pass this ID to `resume` .

- **Follow up on a completed task.** The agent already analyzed something; now you want it to act on that analysis without re-reading files.
- **Recover from a limit.** The first run ended with `error_max_turns` or `error_max_budget_usd` (see [Handle the result](#)); resume with a higher limit. In a single-shot `query()` call the SDK raises after yielding that error result, so catch the error before resuming.
- **Restart your process.** You captured the ID before shutdown and want to restore the conversation.

This example resumes the session from [Capture the session ID](#) with a follow-up prompt. Because you're resuming, the agent already has the prior analysis in context:

Python TypeScript

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

session_id = "..." # The ID you captured in the previous example

async def main():
    # Earlier session analyzed the code; now build on that analysis
    async for message in query(
        prompt="Now implement the refactoring you suggested",
        options=ClaudeAgentOptions(
            resume=session_id,
            allowed_tools=["Read", "Edit", "Write", "Glob", "Grep"],
        ),
    ):
        if isinstance(message, ResultMessage) and message.subtype == "success":
            print(message.result)
```

```
asyncio.run(main())
```

set the `CLAUDE_CONFIG_DIR` environment variable, look under `$CLAUDE_CONFIG_DIR/projects/` instead.

To find your session's directory, replace every non-alphanumeric character in the absolute working directory with `-`: `/Users/me/proj` becomes `-Users-me-proj`. For a working directory whose converted name exceeds 200 characters, Claude Code **truncates the name and appends a hash**, so match the first 200 characters of the converted name when you list `projects/`.

If you set `CLAUDE_CODE_PROJECT_DIR_NAME` beside `CLAUDE_CONFIG_DIR`, look under that name in `projects/` instead. Requires TypeScript Agent SDK v0.3.234 or later, or Python Agent SDK v0.2.140 or later.

You can resume from any working directory:

- **Cross-directory lookup:** Claude Code searches beyond the current project directory to find the ID; see [Resume a session](#) for the exact lookup order and how duplicate copies are handled.
- **Same machine only:** the session file still needs to exist on the current machine.

Before v2.1.223, the lookup was scoped to the current project directory and its git worktrees; SDK versions that bundle an older CLI still behave this way.

To resume sessions across machines or in serverless environments, mirror transcripts to shared storage with a [SessionStore adapter](#).

Fork to explore alternatives

Forking creates a new session that starts with a copy of the original's history but diverges from that point. The fork gets its own session ID; the original's ID and history stay unchanged. You end up with two independent sessions you can resume separately.

- ❗ Forking branches the conversation history, not the filesystem. If a forked agent edits files, those changes are real and visible to any session working in the same directory. To branch and revert file changes, use [file checkpointing](#).

This example builds on [Capture the session ID](#): you've already analyzed an auth module in `session_id` and want to explore OAuth2 without losing the JWT-focused thread. Th



```
session_id = "..." # The ID you captured in the previous example

async def main():
    # Fork: branch from session_id into a new session
    forked_id = None
    try:
        async for message in query(
            prompt="Instead of JWT, outline how OAuth2 would work for the au
            options=ClaudeAgentOptions(
                resume=session_id,
                fork_session=True,
                max_turns=5,
            ),
        ):
            if isinstance(message, ResultMessage):
                forked_id = message.session_id # The fork's ID, distinct fr
                if message.subtype == "success":
                    print(message.result)
    except Exception as error:
        # A single-shot query() raises after yielding an error result. If th
        # failure was an error result, forked_id was already captured by the
        # loop above; connection or process failures yield no result message
        print(f"Session ended with an error: {error}")

    print(f"Forked session: {forked_id}")

    # Original session is untouched; resuming it continues the JWT thread
    try:
        async for message in query(
            prompt="Continue with the JWT approach",
            options=ClaudeAgentOptions(resume=session_id),
        ):
            if isinstance(message, ResultMessage) and message.subtype == "su
                print(message.result)
    except Exception as error:
```

You should see that `forkedId` differs from the original session ID. Resuming the original session still continues the JWT thread, which confirms the fork did not modify the original history.

Resume across hosts

Session files are local to the machine that created them. To resume a session on a different host (CI workers, ephemeral containers, serverless), pick the approach that fits:

- **Pass a session store.** Attach a `sessionStore` / `session_store` **adapter** so the SDK mirrors transcripts to your own backend and another host can resume them. The store lookup key derives from the working directory, so resume from a `cwd` matching the original run's.
- **Move the session file.** Persist `~/.claude/projects/<encoded-cwd>/<session-id>.jsonl` from the first run and restore it inside any directory under `~/.claude/projects/` on the new host before calling `resume`.

Claude Code searches beyond the current project directory to find the ID; see [Resume a session](#) for the exact lookup order and how duplicate copies are handled. Before v2.1.223, the lookup was scoped to the current project directory and its git worktrees; SDK versions that bundle an older CLI still behave this way.

- **Don't rely on session resume.** Capture the results you need (analysis output, decisions, file diffs) as application state and pass them into a fresh session's prompt. This is often more robust than shipping transcript files around.

Both SDKs expose functions for enumerating sessions on disk and reading their messages: `listSessions()` and `getSessionMessages()` in TypeScript, `list_sessions()` and `get_session_messages()` in Python. Use them to build custom session pickers, cleanup logic, or transcript viewers.

Related resources

- **How the agent loop works**: Understand turns, messages, and context accumulation within a session
- **File checkpointing**: Snapshot and revert file changes the agent made within a session
- **Python** `ClaudeAgentOptions` : Full session option reference for Python
- **TypeScript** `Options` : Full session option reference for TypeScript

Was this page helpful?

 Yes

 No

Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

[Trust center](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

[MCP connectors](#)

[Customer stories](#)

[Engineering blog](#)

[Events](#)

[Powered by Claude](#)

[Service partners](#)

[Startups program](#)

[Privacy policy](#)

[Disclosure policy](#)

[Usage policy](#)

[Commercial terms](#)

[Consumer terms](#)